MAJOR PROJECT REPORT
ICT-452

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

computer science and engineering

AI-ExamGaurd

(Computer Based Test(CBT) Platform embedded with AI proctoring(AI ExamGuard))

GURU GOBIND SINGH INDRAPRASTHSA UNIVERSITY

SUBMITTED TO:

Prof. R. Goyal

USICT(GGSIPU)

SUBMITTED BY:

Mohd Fardeen Alam

01416403222

B. Tech-CSE (8th Sem)

Table of Contents

S. No.	Content	Page No.
1	Declaration	
2	Acknowledgement	
3	Introduction	
4	Project Overview	
5	Tools and Technology Used	
6	Implementation & Images	
7	References	

DECLARATION

I, **Mohd Fardeen Alam** (Enrollment No. 01416403222), a student of B.Tech (CSE 8th Semester), USICT, Guru Gobind Singh Indraprastha University, hereby declare that the work which is presented in this Major Project Report entitled "**AI-ExamGuard**" is an original and authentic work of mine under the technical guidance of **Prof.R. Goyal** , USIC&T. I declare that the work in this project has not been submitted in full or in any part for any diploma or degree course of this or any other University to the best of my knowledge and belief. I will be solely responsible myself for any copyright infringement or plagiarism, if any, in the said work, and declare that all necessary due acknowledgement has been made in the content of said work. My supervisor/ guide shall not be held responsible for full or partial violation of copyright or intellectual property rights or any type of plagiarism involved above in the said work.

Mohd FArdeen Alam

01416403222

ACKNOWLEDGEMENT

It is a genuine pleasure to express my profound gratitude and deep regards to my guide "**Prof. R. Goyal**" for his exemplary guidance, monitoring and constant encouragement. I would like to express my special thanks to USICT who gave me the golden opportunity to do this wonderful project on the topic "AI-ExamGuard" (Computer Based Test(CBT) Platform embedded with AI proctoring(AI ExamGuard)) which helped me in doing a lot of learning and I came to understand about so many new things.

Title.

" Computer Based Test(CBT) Platform embedded with AI proctoring(AI ExamGuard)"

Problem Statement

Current Online examination system consist of various problems:

- No Real Time monitoring
- Manual Invigilation dependency is high
- High unfair means chances
- Less Scalable for huge set of students
- High network usage due to live video transmission.

That's why there is a need for intelligent , secure and scalable platform that can ensure exam is conducted properly and efficiently.

Introduction

This project is an online examination platform specifically designed for conducting CBT (computer based test) examination . Current online examination platform lacks monitoring, highly costly to conduct and manage which makes them prone to use of unfair means through phones ,books or tabs switching.Project is divided in three parts **Frontend (UI) , Ai proctoring Layer ,Backend** .This entire project based on **MERN** stack (MongoDB ,Express.js ,React.js ,Node.js). since its open source ,easy to learn and cost effective hence it is the most loved tech stack by the developers to build an entire full-stack project. As we are using express.js the project is scalable and able to handle multiple users request. Proper **authentication** and **authorization** is handled by JWT(Jason web token) and apart from this **Tensorflow.js , MediaPipe** are used for image detection and recognition which continuously monitors user and its behaviour through inbuilt camera or webcam.

Objective

- Provide a platform for conducting online examination.
- Easy to use and user friendly responsive UI.
- Secure back-end for exam integrity.
- Store exam data ,result data efficiently for fast retrieval.
- Role based frontend i.e user or examiner.
- Scale for large no of students.

Scope of Project

- Student /Examiner registration and authentication.
- Timely and Efficient Exam Conduction.
- Automated evaluation tests.
- Real Time Ai Monitoring.
- Suspicious Activity Logging.
- Examiner Dashboard for managing Tests and examinee.

Project Architecture

Project is based on MVC (Model-Views-Controller) architecture .It is the most used industry standard framework [1] .It separates ui/ux (front-end) part into **Views** , business logic (back-end) and database schema (back-end) into **Model** and **Controller** acts as a bridge between model and views.

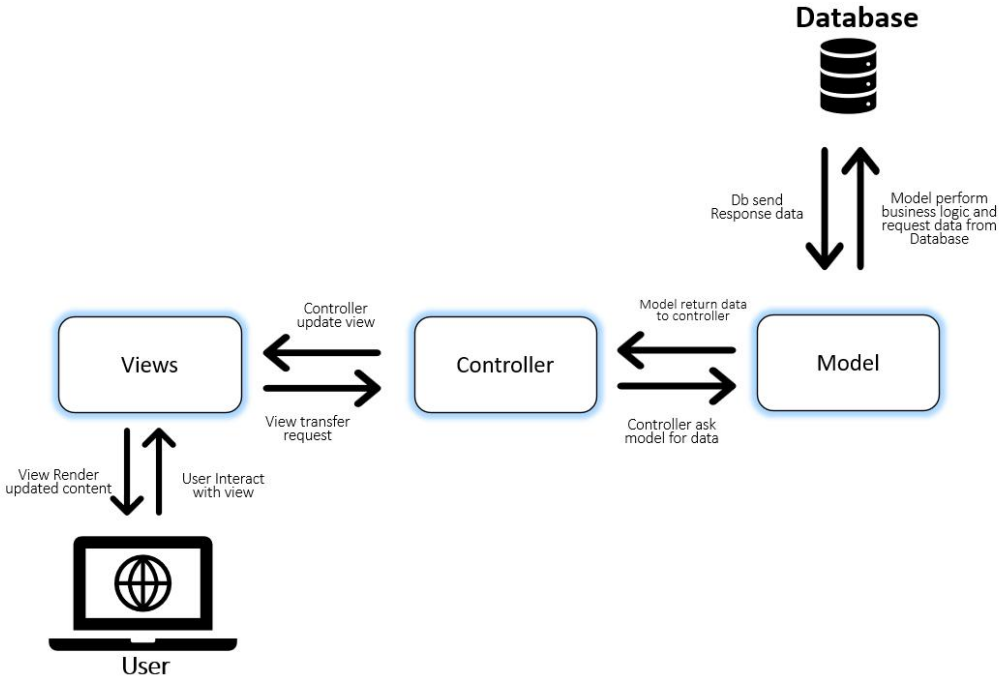
Exam Interface and monitoring through AI is handled in frontend.

AI model locally process the data in browser through webcam.

If any suspicious activity are generated those are stored into database.

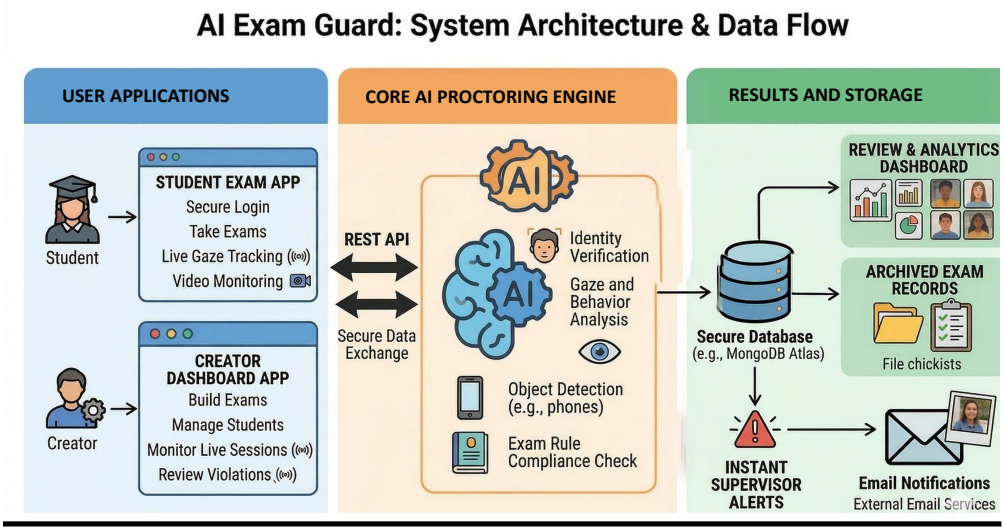
All exam Result are stored though back-end into MongoDB

- This makes managing and updating code efficient.
- Tracking bugs easier .
- Increases cohesion and reduces coupling.



MVC Architecture Diagram

Project Flow



Module Breakdown

•**Authentication Module** — JWT-based login and registration for both creator and student roles. Role-scoped tokens stored under separate localStorage keys enable simultaneous sessions.

•**Test Creator Module** — Multi-section test builder with per-question MCQ and numerical types, marks, negative marking, section timers, and strict mode configuration. Full CRUD with publish/unpublish toggle.

•**Enrollment Module** — Students join tests using alphanumeric codes. Enrollment records link student accounts to specific tests and are used to populate the Student Dashboard.

•**Exam Engine Module** — Fullscreen-locked exam interface with real-time total and section timers, strict section navigation, question palette, answer state management with ref synchronization, and beforeunload beacon for safe submission.

•**AI Proctoring Module** — TensorFlow.js and COCO-SSD models running client-side on sampled camera frames. Violations dispatched to a shared handleDetect callback for categorization and display.

•**Scoring Module** — Server-side score calculation with positive and negative marking per question. Returns score, percentage, totalMarks, passed, and timeTaken on the Attempt document.

•**Analytics Module** — Per-test results panel with score band distribution, pass rate, average score, and per-student attempt history with violation summaries.

•**Deployment Module** — GitHub-connected CI/CD with Vercel for frontend and Railway for backend. MongoDB Atlas for managed cloud database. Environment variables for all secrets.

Tools and Technologies

1.React.js – React is a front-end **library** used for designing user friendly and responsive websites. This is **component** [2] **based** library which combines functionalities of html and java-script code .Due to its component based same codes can be reused multiple times .

2.Tailwind- it is built on top of the CSS and is a **utility-first** [3] css framework which provides premade classes for css and can be used directly into html code without leaving html file.

3. Node.js – It provides **javascript runtime environment** which lets us run javascript code outside the web browser. Its Built on top of the google chromes V8 engine and lets us use javascript for creating servers and web application.

4.Express.js – Express is a minimal and flexible framework [4]for creating servers and web application and it is built on top of node.js which makes creating and handling more easier.

5.MongoDb- Since industrial data transferring format is json Mongodb lets us store data into JSON format i.e with **key value pair** for better storage and retrieval.

6.JWT- To enhance **authentication** and **authorization** we use jsonwebtoken which creates a **digitally signed tokens** [5] used for transmitting information between client and server.

7.TensorFlow.js - A Machine learning Library[6] developed by Google in 2015 and is open source to use . It lets the user build and train AI models using neural network and lets the user run theese model on browser.

8.MediaPipe- It is framework also developed by google and its open source ,its is used to **process real time multimedia**[7] data for example **audio ,image ,video or sensor inputs.**

9.Face-api.js - Its an javascript library built on tensorflow.js and it allows face recognition and analysis.

10.COCO-SSD - It's a pre trained model used to detect objects in images or videos.

11. Vercel – It is a cloud based **hosting service** lets us build and **deploy** our entire web application and make them available and scalable over the internet .Its a paid service but we can still get small storage for hosting our application on vercel

References

[1] MVC Architecture :- https://www.geeksforgeeks.org/software-engineering/mvc-framework-introduction/

[2] React docs - https://react.dev/

[3] Tailwind- https://tailwindcss.com/

[4] Express - https://expressjs.com/

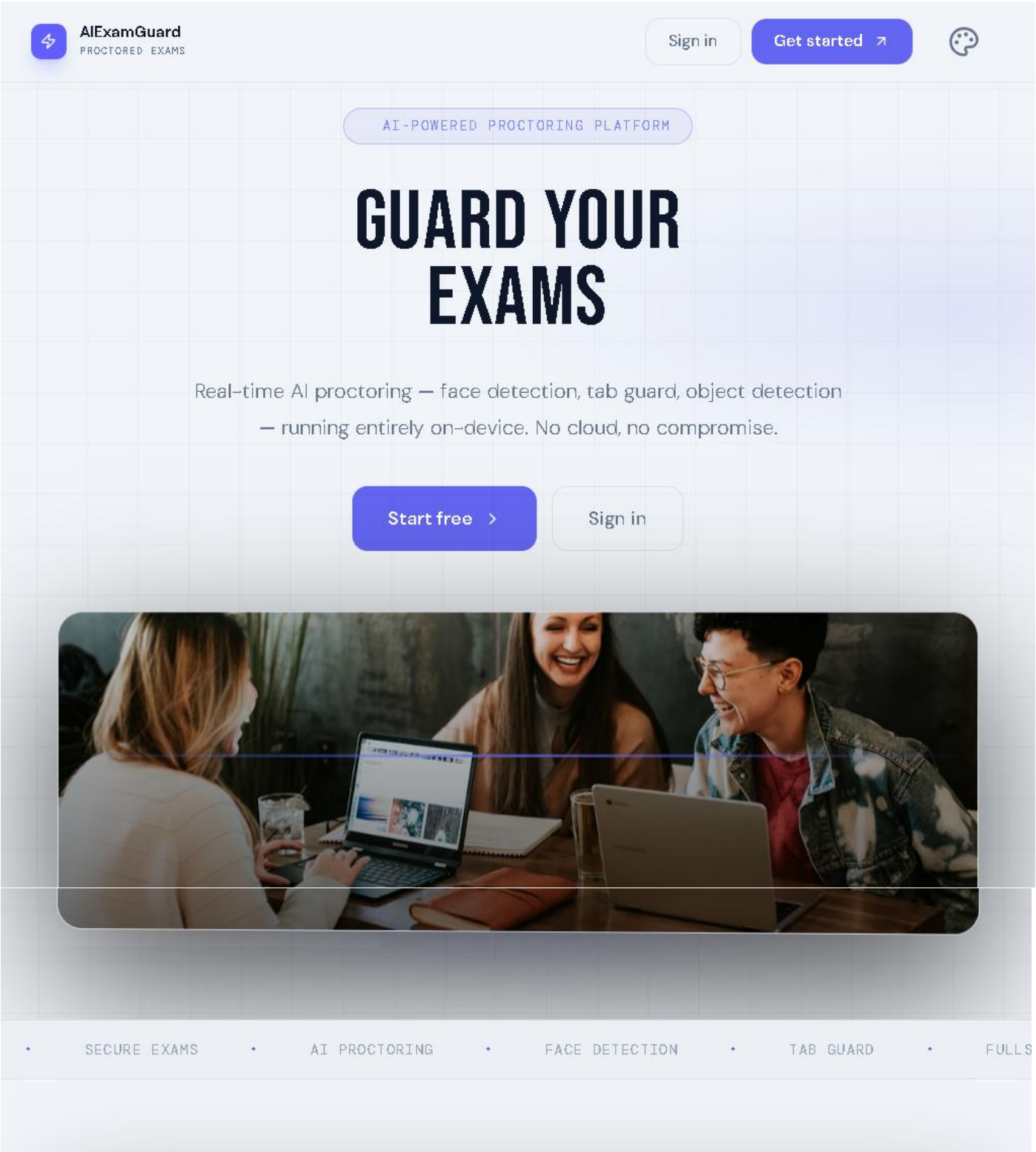
[5]JWT - https://www.jwt.io/introduction#what-is-json-web-token

[6]TensorFlow - https://www.tensorflow.org/js/tutorials

[7]MediaPipe - https://ai.google.dev/edge/mediapipe/solutions/guide

Images of the User Interface (Frontend)

1. Landing Page/Home Page of the Website



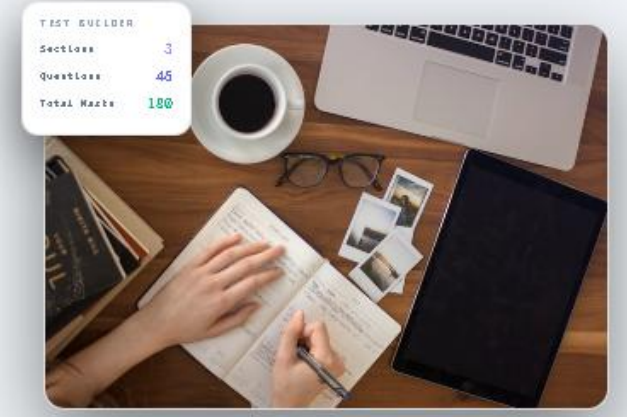
FOR STUDENTS

FOCUS ON THE EXAM

Clean, distraction-free interface. Navigate questions freely within each section, track your answers visually, and submit with confidence. The AI handles security — you handle the answers.

- Section-wise timed questions with live countdown
- Visual question map — see answered at a glance
- MCQ and numerical answer types supported
- One-click submit with instant confirmation

Register as Student



FOR CREATORS

BUILD TESTS IN MINUTES

A three-panel editor that gets out of your way. Add sections, set timers, write MCQ or numerical questions, attach images, and marks auto-calculate. Publish with one click.

- Unlimited sections with individual timers
- MCQ with images + numerical answer types
- Auto marks calculation across all sections
- Publish, draft, edit and delete anytime

Start creating

UNDER THE HOOD

BUILT FOR INTEGRITY

Face Tracking

Continuous face detection flags absence or multiple people in real time.

Device Detection

On-device AI spots phones, books and prohibited items without cloud upload.

Fullscreen Lock

Enforced fullscreen with instant re-entry prompts and violation logging.

Tab Guard

Every tab switch is caught, logged, and triggers an immediate warning.

Section Timers

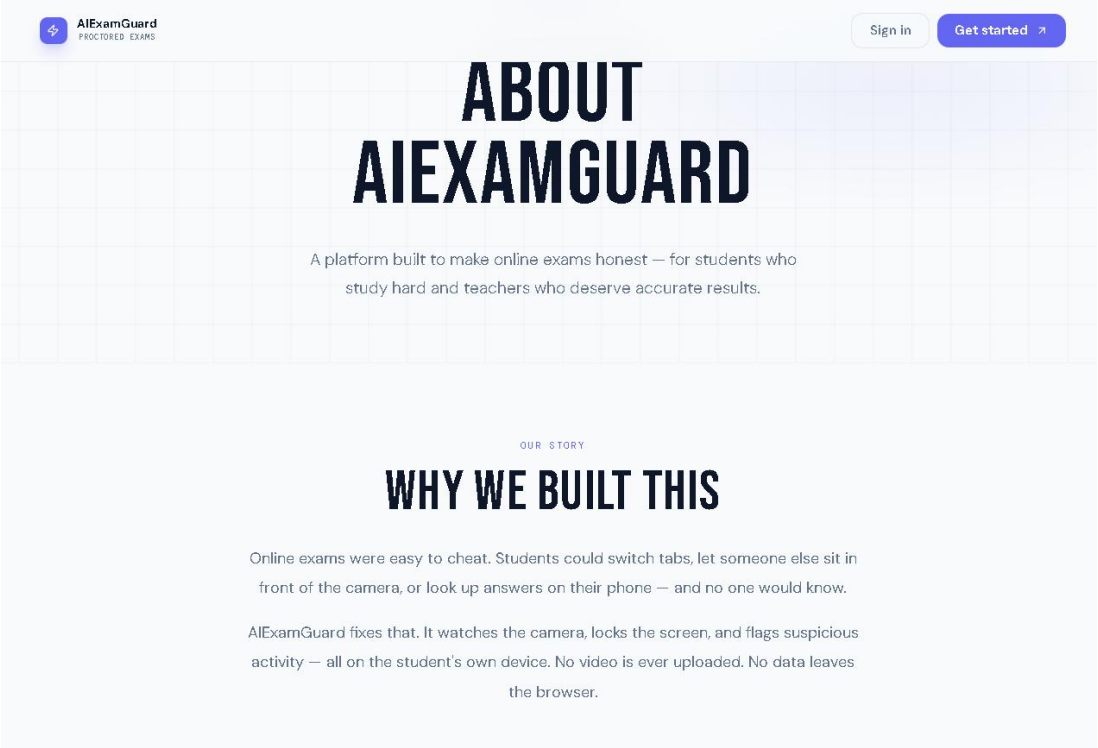
Per-section countdowns auto-advance students — fair for everyone.

Private by Design

Zero video upload. All AI runs locally. Your data never leaves the device.

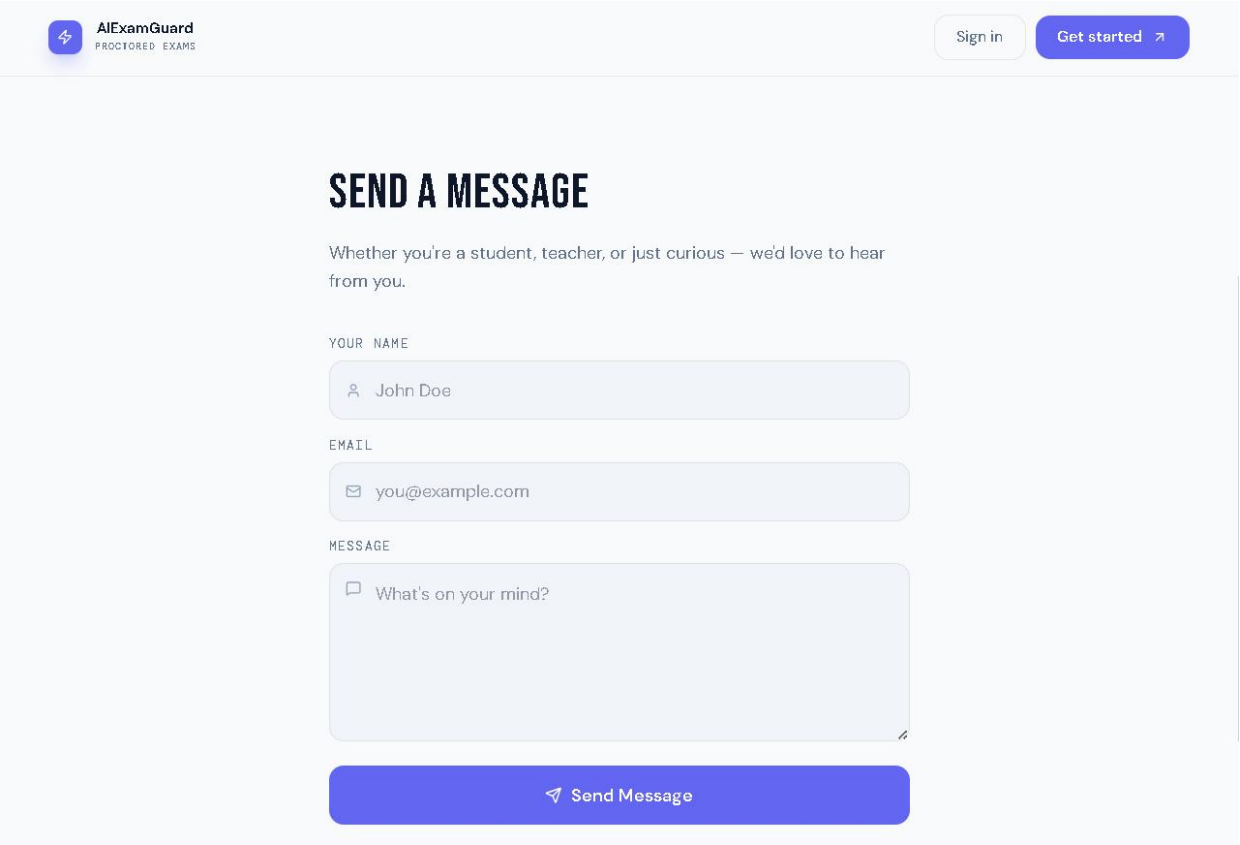
2. About Page

It’s a basic about page of the website telling its story ,goal and mission.



3. Contact Us Page

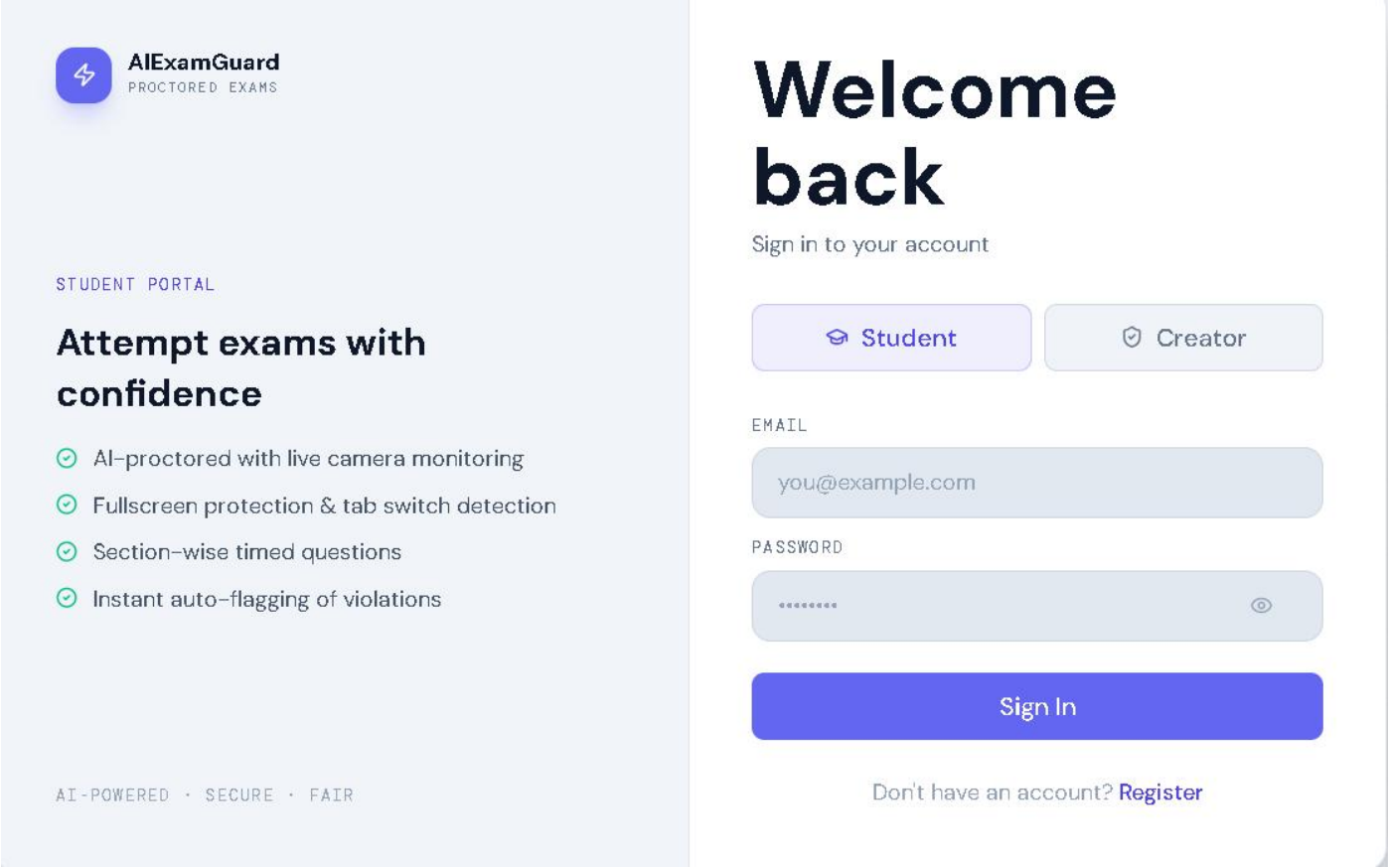
It’s a basic contact page of me from which users can contact me through email .



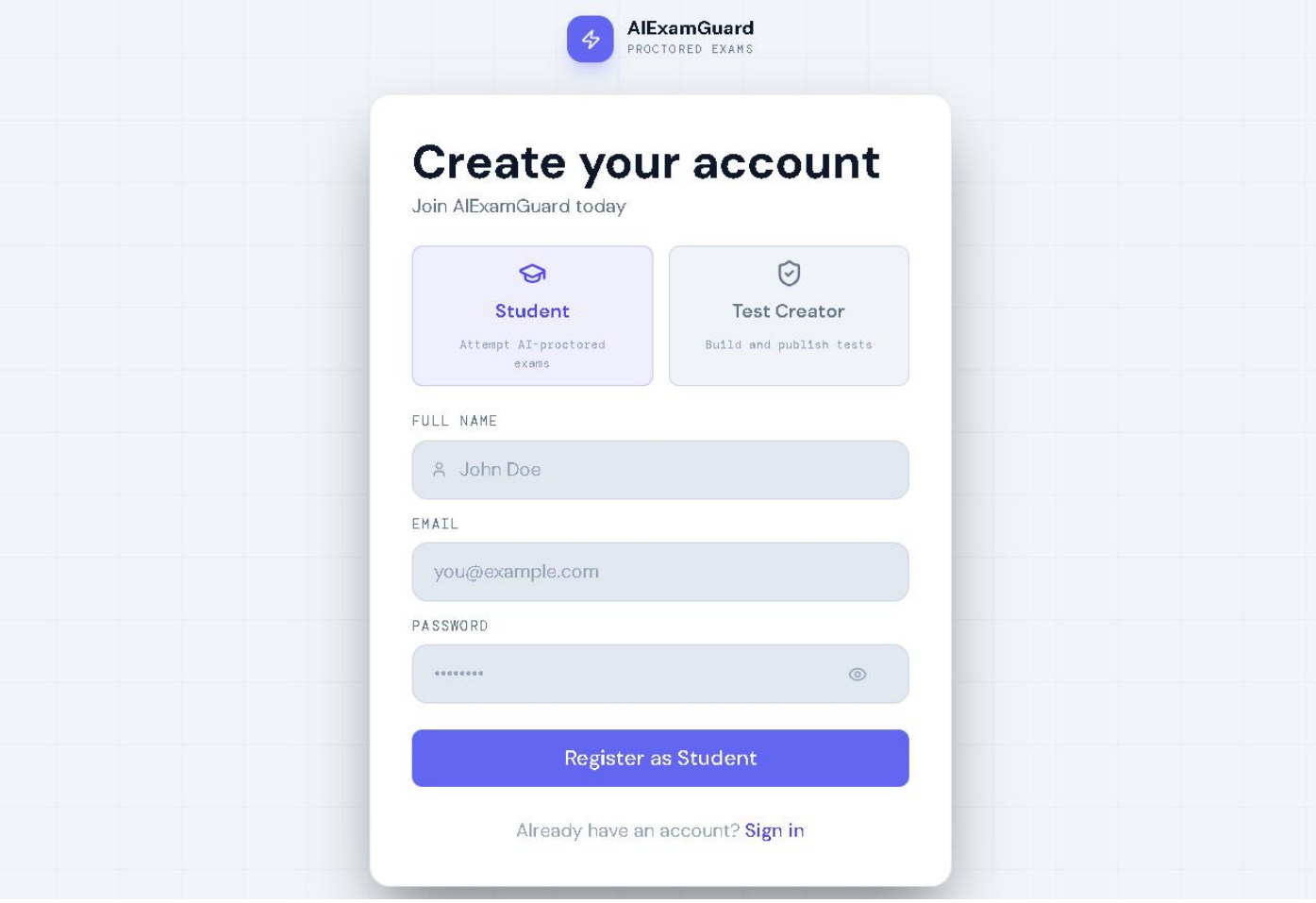
4.Login /Sign-Up - Every website needs to maintain info about their users for better user experience and to identify users to do which uneeds to sign up and then can log in to website

-

Login



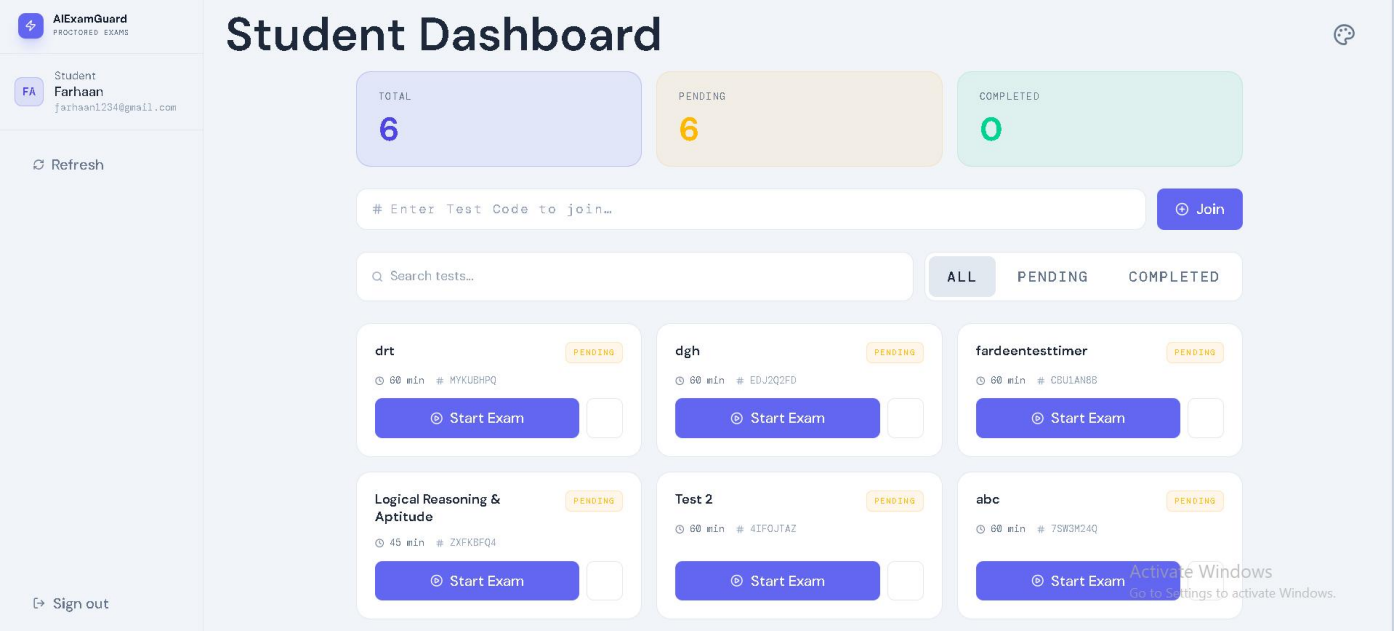
Sign Up


AIExamGuard
PROCTORED EXAMS
Create your account
Join AIExamGuard today
Student
Attempt AI-proctored exams
Test Creator
Build and publish tests
FULL NAME
John Doe
EMAIL
you@example.com
PASSWORD
Register as Student
Already have an account? Sign in

5.Dashboards - This is the main page from which a student can join test and creator can manage tests and students

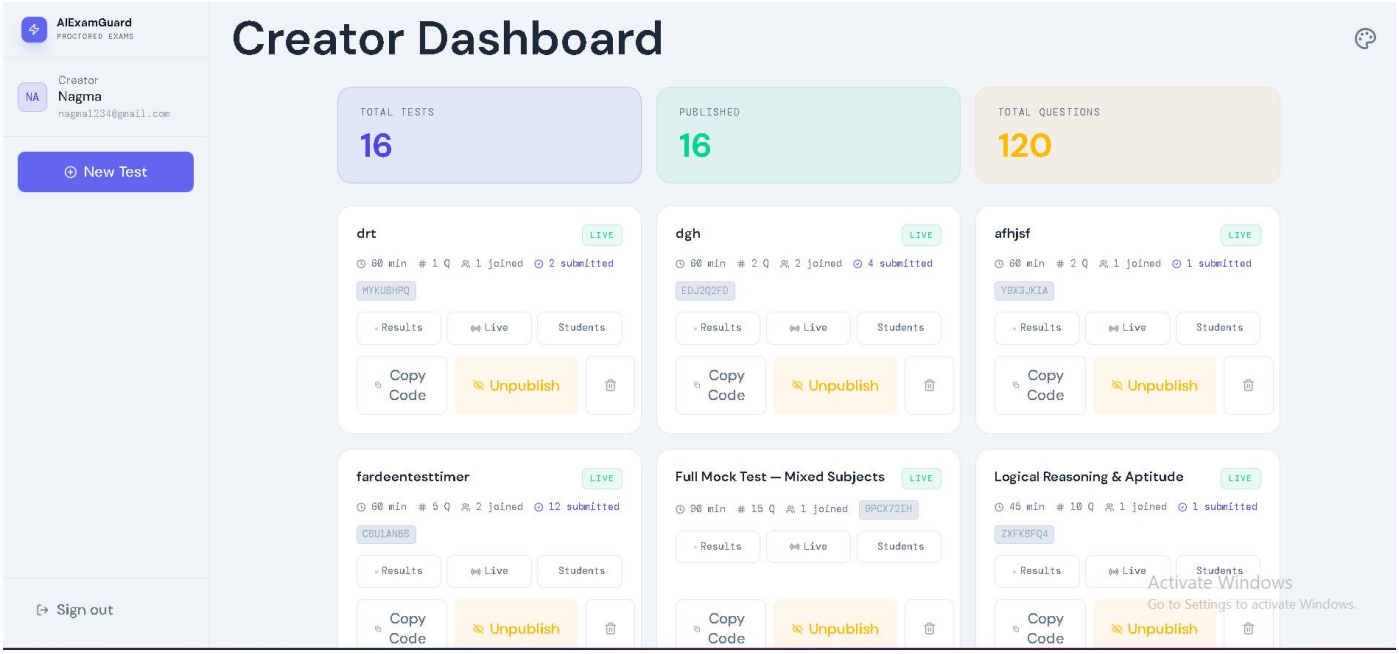
Student Dashboard Page

In student dashboard student can join tests through test shared by creator and can start attempting the test by clicking start exam



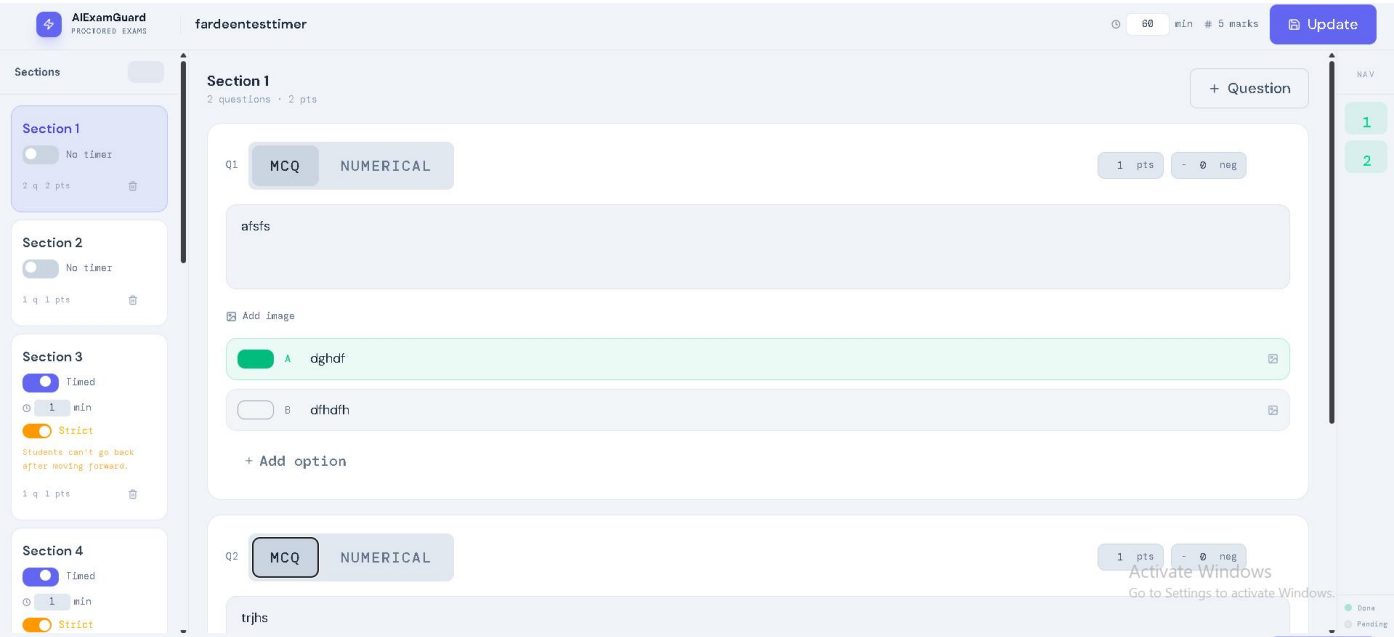
Creator Dashboard Page

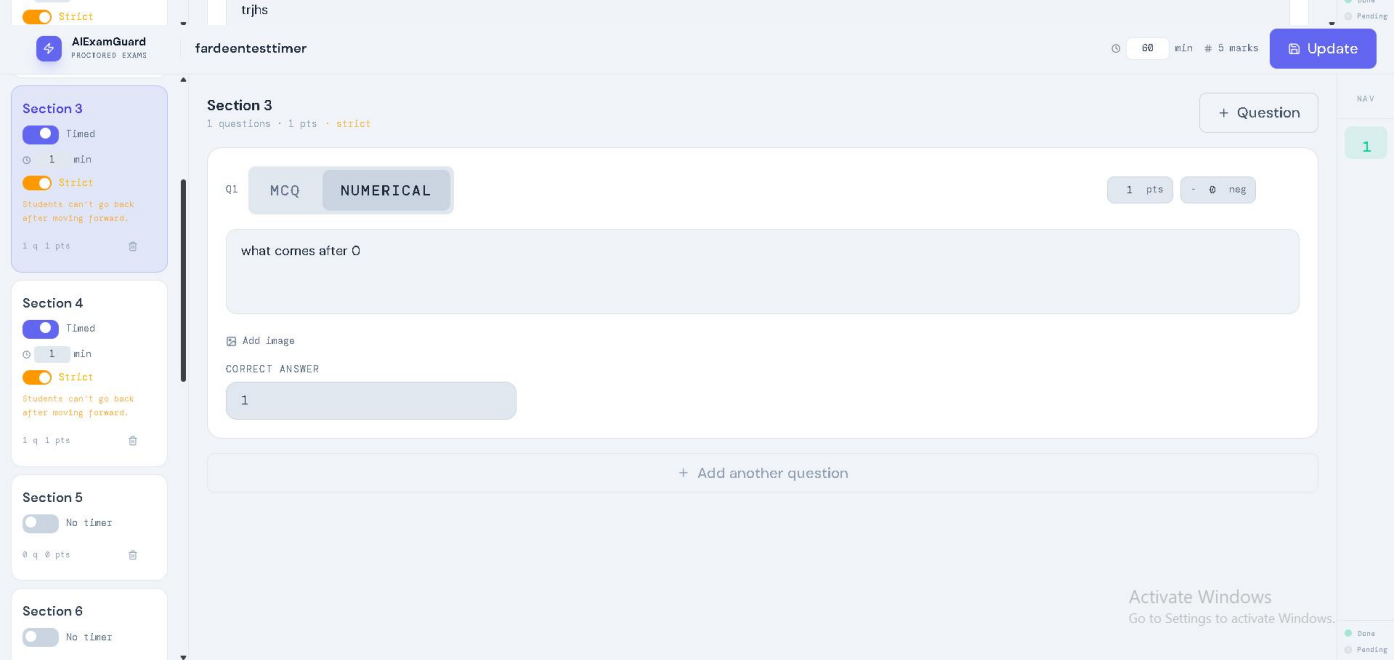
In creator dashboard creator can create tests new test by clicking new Test button or can update existing test by clicking upon the test itself also creator can see who attempted test,their live monitoring and their result



6.Test Creation Page -

This page lets the user create tests and add infinite no of questions and sections , each question can be of two types mcq and numerical type creator could also add image in questions as well as in mcq options .

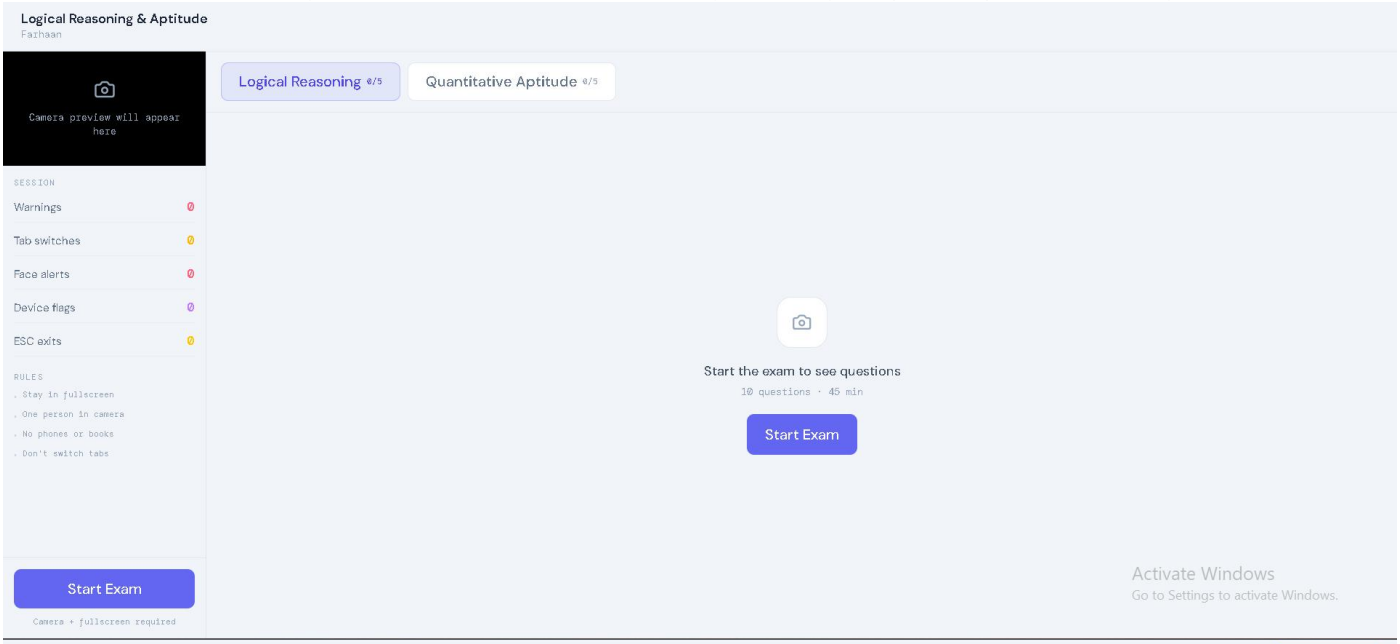


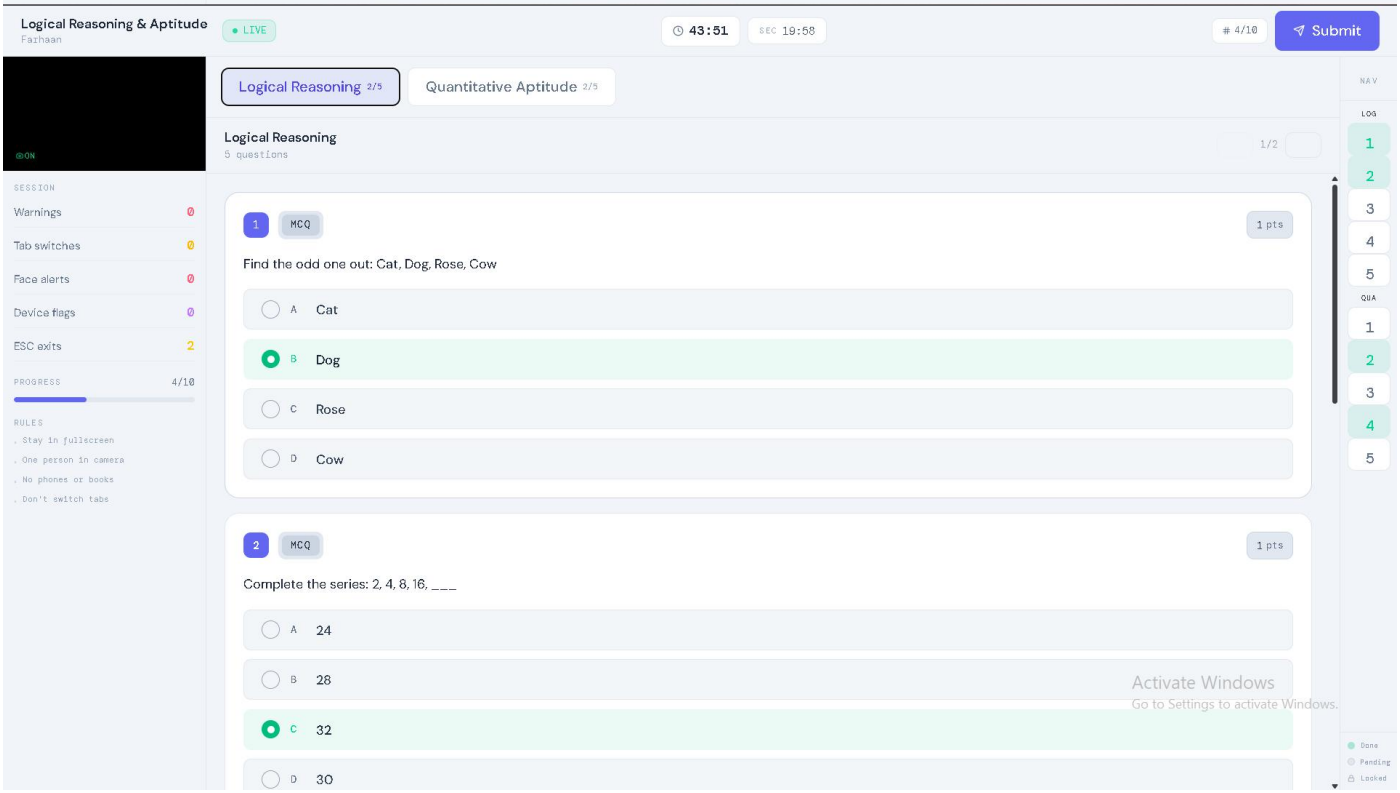


Creator can edit test title , creator can edit sections and can set timers for each sections as well creator could lock certain sections so that user cannot switch sections after coming to locked sections.

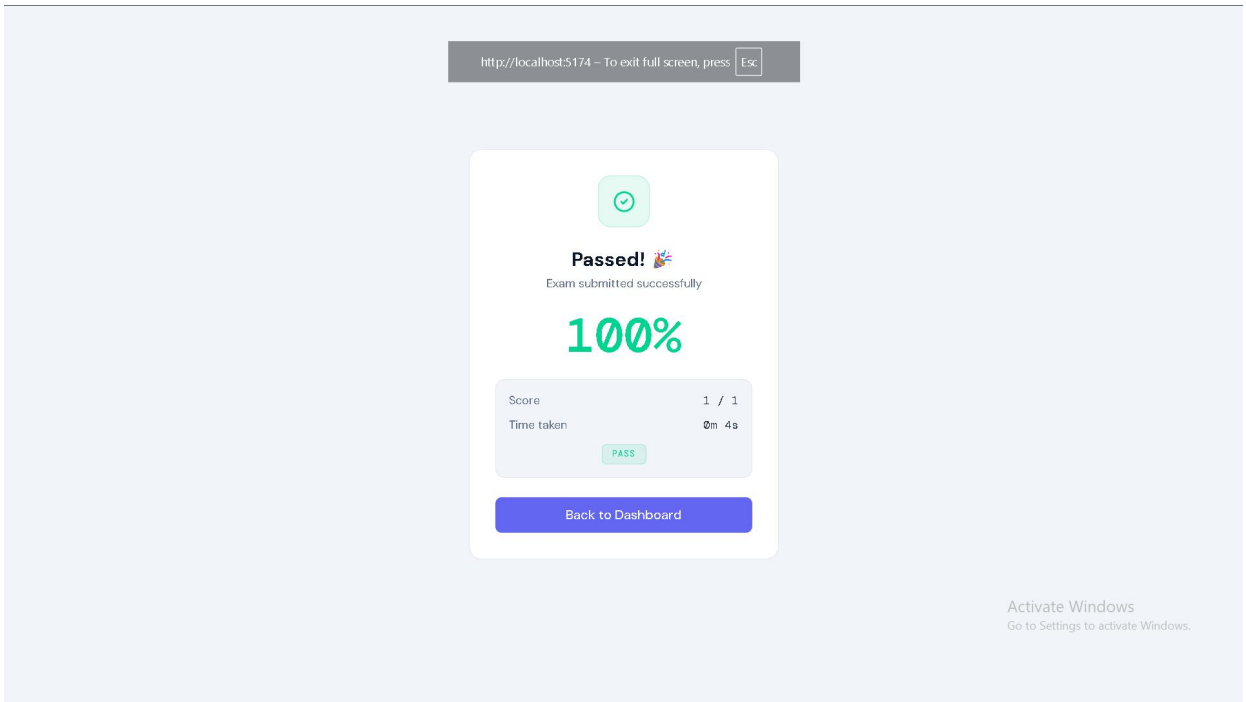
7.Exam Page -

This page is the backbone of the entire system upon which student spends most of their time by attempting test generated by the creators .

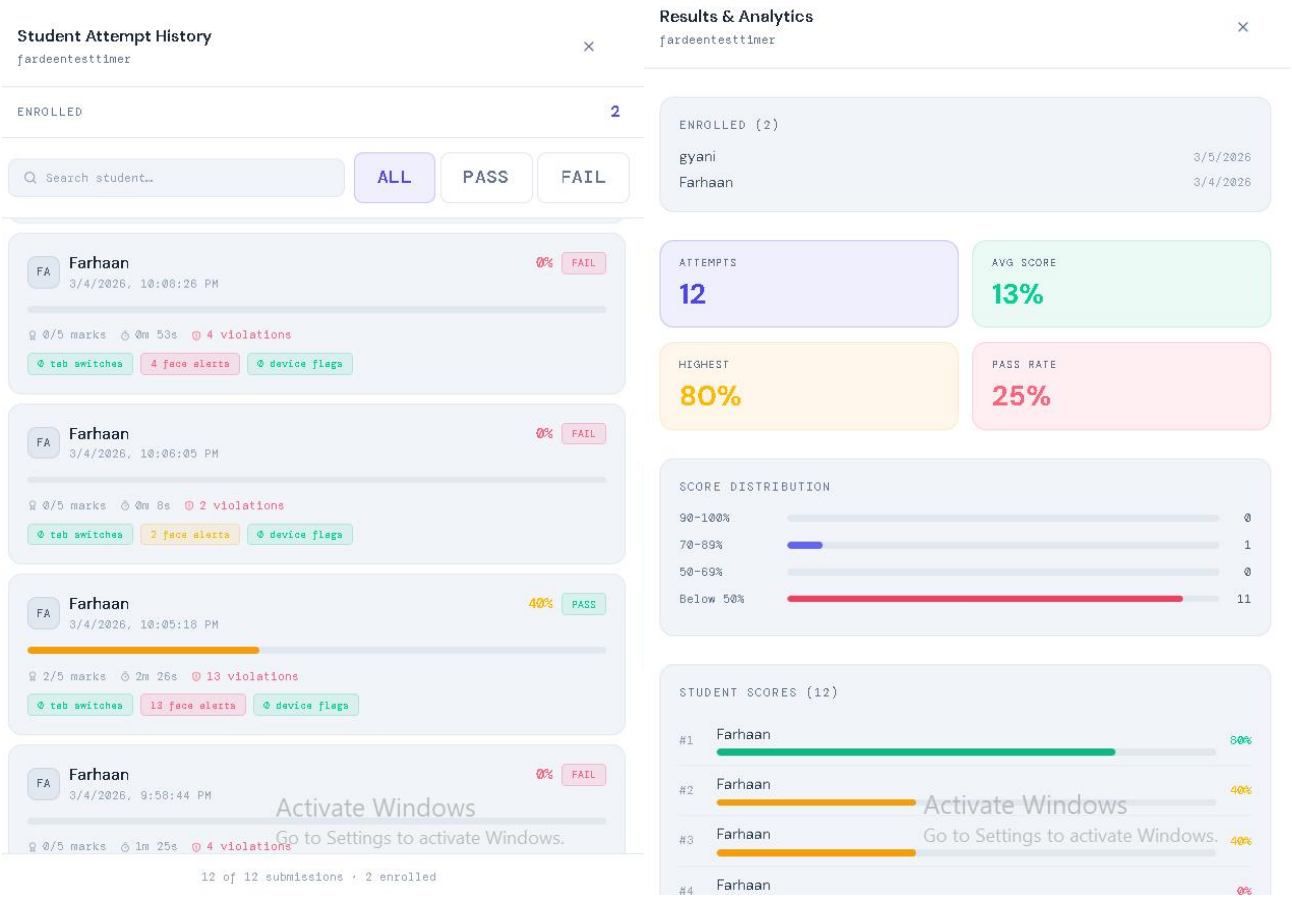




Result Pop-Up after Submitting the Test

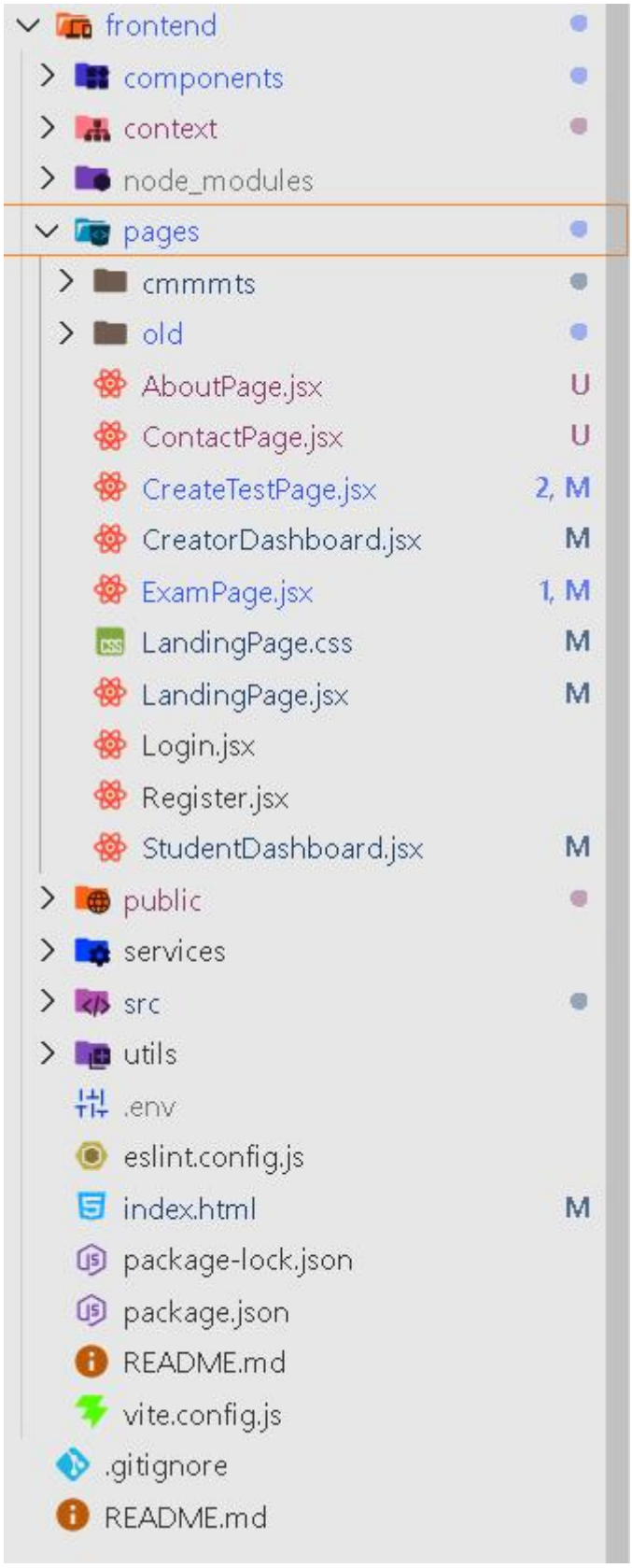


8. Result and Attempts Page (Only For Creator to see)



Code for Frontend

1. Frontend Project Structure



Main -- App.jsx

frontend > src > App.jsx > App

```jsx
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import ProtectedRoute from "../components/ProtectedRoute";
import LandingPage from "../pages/LandingPage";
import Login from "../pages/Login";
import Register from "../pages/Register";
import AboutPage from "../pages/AboutPage";
import ContactPage from "../pages/ContactPage";
import CreatorDashboard from "../pages/CreatorDashboard";
import StudentDashboard from "../pages/StudentDashboard";
import ExamPage from "../pages/ExamPage";
import CreateTestPage from "../pages/CreateTestPage";

export default function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          <Route path="/" element={<LandingPage />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/about" element={<AboutPage />} />
          <Route path="/contact" element={<ContactPage />} />
          <Route path="/creator-dashboard" element={<ProtectedRoute role="creator"><CreatorDashboard /></ProtectedRoute>} />
          <Route path="/create-test" element={<ProtectedRoute role="creator"><CreateTestPage /></ProtectedRoute>} />
          <Route path="/create-test/:testId" element={<ProtectedRoute role="creator"><CreateTestPage /></ProtectedRoute>} />
          <Route path="/student-dashboard" element={<ProtectedRoute role="student"><StudentDashboard /></ProtectedRoute>} />
          <Route path="/exam/:id" element={<ProtectedRoute role="student"><ExamPage /></ProtectedRoute>} />
          <Route path="*" element={<Navigate to="/login" replace />} />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}
```

Main.jsx

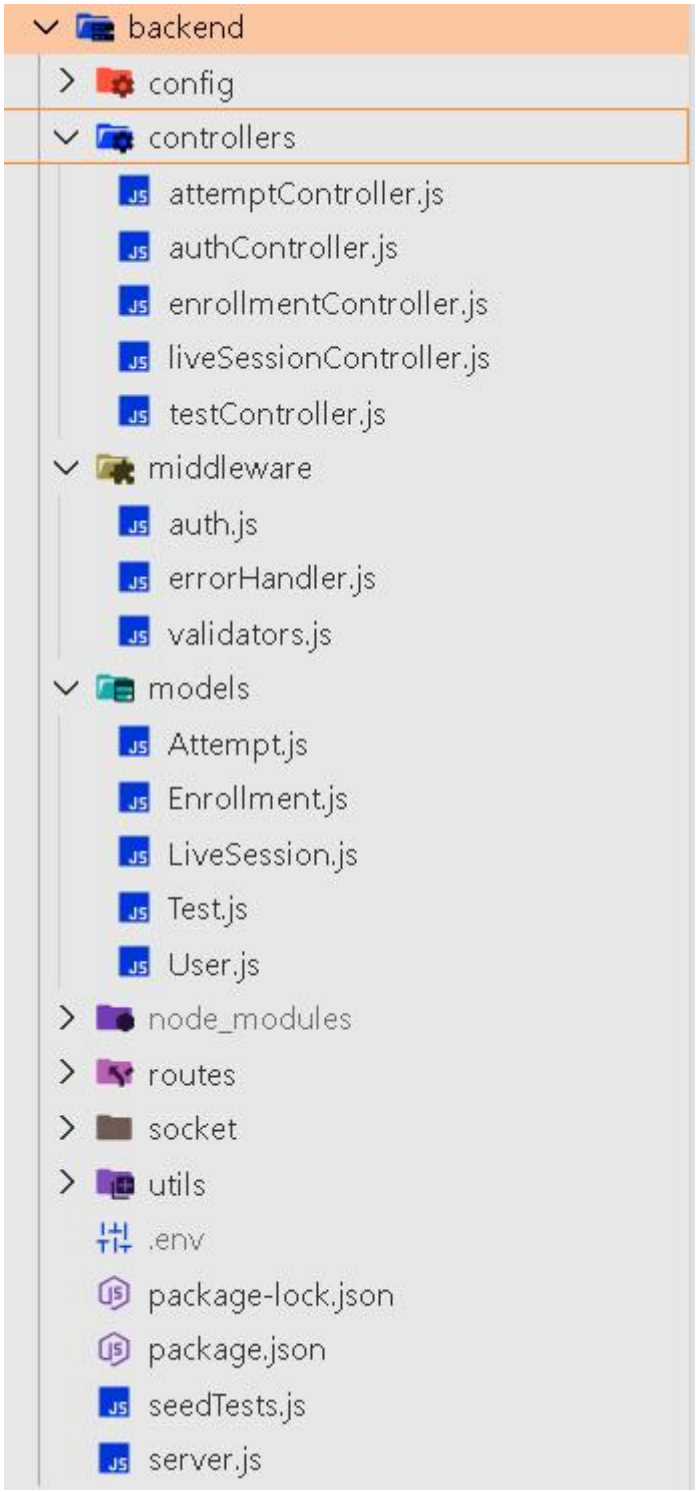
frontend > src > main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import { ThemeProvider } from "../context/ThemeContext";
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <ThemeProvider>
    <StrictMode>
      <App />
    </StrictMode>
  </ThemeProvider>
)
```

Complete Backend

1. Project Structure of Backend



2. Server.js - Entry point for all the request to backend

backend > server.js > ...

```javascript
require("node:dns/promises").setDefaultResultOrder("ipv4first");
require("node:dns/promises").setServers(["8.8.8.8", "1.1.1.1", "8.8.4.4"]);
require("dotenv").config();
const express      = require("express");
const cors         = require("cors");
const connectDB    = require("./config/db");
const errorHandler = require("./middleware/errorHandler");
connectDB();
const app = express();
app.use(cors({
  origin:      [process.env.CLIENT_URL || "http://localhost:5173", "http://localhost:5174"],
  credentials: true,
}));
app.use(express.json({ limit: "10mb" }));
app.use(express.urlencoded({ extended: true }));
app.get("/api/health", (req, res) => {
  res.json({
    success: true,
    message: "AIExamGuard API is running",
    env:     process.env.NODE_ENV,
    time:    new Date().toISOString(),
  });
});
app.use("/api/auth",        require("./routes/authRoutes"));
app.use("/api/tests",       require("./routes/testRoutes"));
app.use("/api/enrollments", require("./routes/enrollmentRoutes"));
app.use("/api/attempts",    require("./routes/attemptRoutes"));
app.use((req, res) => {
  res.status(404).json({ success: false, message: `Route ${req.originalUrl} not found` });
});
app.use(errorHandler);
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`🚀  Server running in ${process.env.NODE_ENV} mode on port ${PORT}`);
});

module.exports = app;
```

Activate Windows
Go to Settings to activate Windows.

3. MongoDB Database

MongoDB Compass - suzane.fploc.mongodb.net/CBT_platform.tests

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

- suzane.fploc.mongodb.net
  - CBT_platform
    - attempts
    - enrollments
    - livesessions
    - tests
    - users
  - Ecommerce
  - User_Authentication
  - admin
  - image_uploader
  - local
  - sample_mflix
  - test
  - urlShortener

tests

suzane.fploc.mongodb.net > CBT_platform > tests

Open MongoDB shell

Documents 16 | Aggregations | Schema | Indexes 2 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | UPDATE | DELETE | EXPORT DATA | EXPORT CODE

25 | 1 - 16 of 16

```
_id: ObjectId('69a8321a6b01d6db761a00e9')
title : "Computer Science — Basics"
description : "Covers data structures, algorithms, networking and OS concepts."
createdBy : ObjectId('69a58fbda3c050e6e5868dc0')
duration : 50
sections : Array (2)
  0: Object
    title : "Programming & Data Structures"
    duration : 1500
    questions : Array (5)
      0: Object
        type : "mcq"
        text : "Which data structure uses LIFO?"
        image : null
        options : Array (4)
        correct : "Stack"
        marks : 1
        negativeMark : 0.25
        _id : ObjectId('69a8321a6b01d6db761a00eb')
      1: Object
      2: Object
      3: Object
      4: Object
    _id : ObjectId('69a8321a6b01d6db761a00ea')
  1: Object
published : true
settings : Object
createdAt : 2026-03-04T13:22:34.205+00:00
```

Activate Windows
Go to Settings to activate Windows.